

approach and repeats.

Such software could also work like a factory, with inputs and outputs, and a monitor that manages the factory. In case the output is not the best, the input can be tweaked to make it so.

Send In The Agents

Today's software works by reacting to users. To get an application to work just the way you want it, you need to give it specific instructions. The average human, however, is quite averse to such activity. When we give instructions to other humans, we hardly spell out a detailed plan of action to follow. Instead, we tell them what would make us happiest, and then leave them to their devices. And while they set out to get us what we want, they do so independently.

The key to adaptive software is the *agent*—a piece of software designed to help you meet your goals. Just like human agents, all they need is an idea of your *preferences* rather than your concrete goal. They will then act autonomously on your behalf and interact with other agents for an output that best suits your preferences. In addition, they keep an eye out for what's going on in their environment, and can take the initiative to correct the consequences of, say, a network failure.

One thing that might concern us, then, is how to prevent this agent from running wild and wreaking havoc—we need to know that the agent is doing what we want, and not just what *it* thinks we want. The answer sounds quite simple—teach it! Once we've told the agent our preferences, we need to let it have a few practice

The key to adaptive software is the agent—a piece of software designed to help you meet your goals

runs to see how it's doing, and tell it where it's going wrong. To do this, we apply a technique called *Reinforcement Learning*.

Teach Me, Master

Reinforcement Learning (RL) relies on the concept of *rewards* to tell agents whether they have hit upon an optimal solution or not. The goal of the agent, like all of us, is to maximise the rewards of performing a task.

To understand RL, let us build ourselves a city, and throw in an automated bus system. Passengers would approach a bus-stand, press a button, and soon a bus would be on its way to pick them up.

Let us look at just one of these buses. This bus now has to pick up various passengers across the city and drop them to their destinations. Only once the bus is empty can it turn back and go the other way. What is the best way to go about it? Nobody really knows yet. What we need to do is establish a *policy* that will define the best way for the bus to fulfil all its duties.

To define our policy, we now need to decide *why* the “best” way would be the best. For this case, let's say that our criteria are fuel efficiency (because we are all environmentally) and ensuring that a maximum number of passengers reach their destinations without spending too much time waiting for the bus. Now let's set up a reward system. Let's say that the bus gets -5 for running empty (wasting valuable fuel), -1 for each bus-stop with a waiting passenger, and +5 for each passenger who reaches his destination.

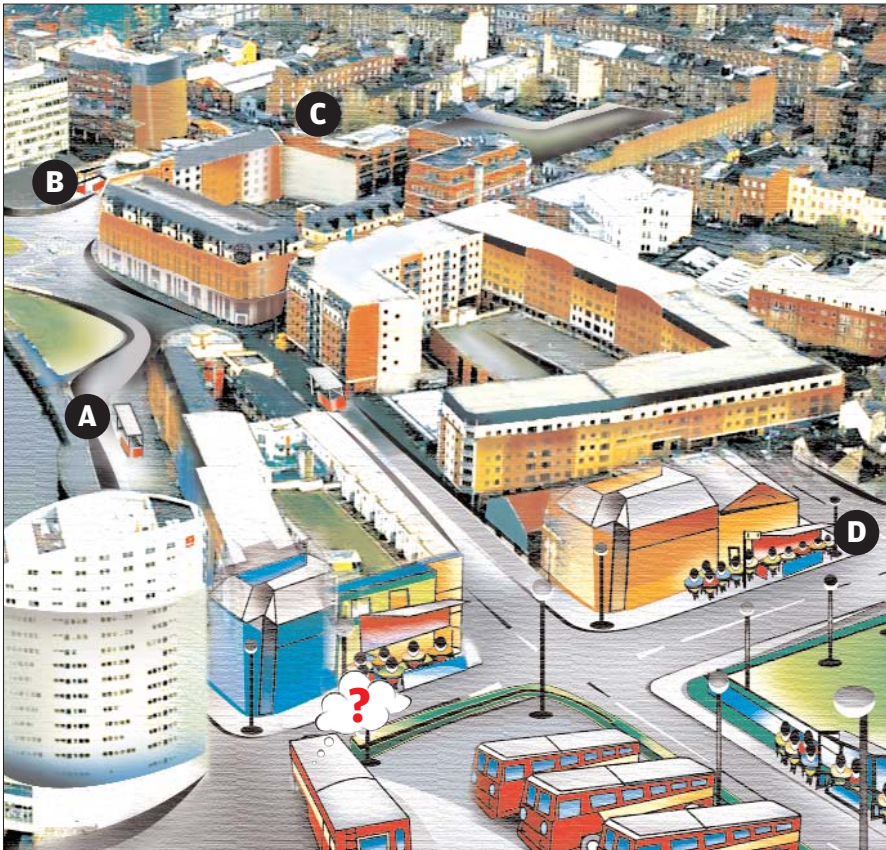
Now, writing software to prepare for each scenario possible above is a mammoth task for programmers. And even after that, if the environment changes—the aging bus might get slower—the programmers need to be called in again. And after that, if we were to set up another futuristic city with this same system, we'd have to turn to the programmers again.

RL is the answer to this problem because it *trains* the system to do its job rather than have it pre-programmed. This means that if any part of the environment changes, the system will be re-trained all on its own without programmer intervention.

But before we send our software agent to train itself, we need to equip it with some vital information. Firstly, it needs to know the *reward function*—how rewards are calculated—so that it can decide which course of action would get it the maximum reward.

Still, merely knowing the immediate rewards of an action is not enough. The agent also needs to know the long-term rewards of a series of actions. Consider this: suppose there are passengers at bus-stops A, B and C and each of them wants to go to bus-stop D.

The agent might just calculate that there are high rewards in taking A to D. But then, it would have to return for B. The logical choice, we can



The bus's dilemma: which way is the most rewarding?